style the elements:

```
strongStyle:
  fontWeight: 'bold'
  color: 'red'

strongElements = document.getElementsByTagName 'strong'
for strongElement in StrongElements
  for styleName, styleValue of strongStyle
    strongElement[styleName] = styleValue
```

With the object syntax, you very quickly arrive at syntax very similar to CSS. The limited vocabulary and small number of features make CSS a DSL. Although CSS is an entire language (an *external* DSL), a mini-language with a limited vocabulary is often useful in other contexts where an entire standalone language would be too time-consuming to implement. For those cases, you can create internal DSLs.

8.2.2. Internal DSLs

One well-known example of utilizing a DSL is the popular jQuery framework. Instead of directly using the APIs designed for DOM manipulation, jQuery provides a DSL that simplifies things. Another common use of an internal DSL is in a testing framework. For example, suppose you want to test if an array contains a particular item. Using only the `assert` module for Node.js works, but you might not find it expressive enough:

```
assert = require 'assert'

haystack = [1..900]
needle = 6
assert needle in haystack
```

The built-in assert module throws an exception when an assertion fails. When it succeeds, as it did here, then there's no output (the output is undefined).

How about testing that a string contains another string?

```
assert 'fundamental'.indexOf('fun') >= 0
```

That works too. But all your tests are phrased in terms of the assertion library, which might not be very comprehensible to you when writing or